



Ministère de l'Enseignement Supérieur
de la Recherche Scientifique et de
l'Innovation
Université Sultan Moulay Slimane
L'École Supérieure de Technologie
Fkih Ben Salah



Rapport de Projet de Fin d'Études

Présenté en vue de l'obtention du
Diplôme Universitaire de Technologie
Filière : Génie Informatique

Présenté et Soutenu par :

Mr. Mohsine DAHNAOUI,

**Mr. Anas SABIQ,
Mr. Ilyas QUIOAMI**

Encadré par :

Professeur : Mr. Khalid QBOUCHE

**Application web de réservation
intelligente (hôtel) (Laravel/vuejs)**

Soutenu le **15/06/2026** devant la commission d'examen composée de :

Pr. Prénom NOM Encadrant

Pr. Prénom NOM Examineur

Pr. Prénom NOM Président

Année universitaire **2024-2025**

Dédicace

*À nos parents,
pour leur soutien inconditionnel et leurs sacrifices
qui ont rendu ce parcours possible.*

*À nos professeurs de l'École Supérieure de Technologie,
qui nous ont transmis les connaissances et la passion
pour l'informatique.*

*À notre encadrant Pr. Khalid QBOUCHE,
pour ses conseils précieux et son accompagnement
tout au long de ce projet.*

À tous ceux qui croient en l'innovation au service de l'éducation.

Remerciements

Nous souhaitons adresser nos sincères remerciements à l'École Supérieure de Technologie Fkih Ben Salah pour nous avoir donné l'opportunité de mener à bien ce projet de fin d'études.

Nos remerciements s'adressent tout particulièrement à **Pr. Khalid QBOUCHE**, notre encadrant pédagogique, pour son accompagnement rigoureux, ses conseils précieux et sa disponibilité constante durant toute la durée de ce travail.

Nous exprimons également notre reconnaissance à l'ensemble de l'équipe pédagogique du département Mathématiques et Informatique pour la qualité de leur enseignement et leurs orientations expertes qui ont considérablement enrichi notre approche.

Nous remercions chaleureusement tous ceux qui ont contribué à faire de cette expérience un parcours enrichissant.

Résumé

Le présent rapport détaille le développement de **Grand Palais**, une application web de réservation hôtelière intelligente réalisée dans le cadre de notre projet de fin d'études au département Mathématiques et Informatique de l'École Supérieure de Technologie Fkih Ben Salah. Ce projet répond à la problématique de digitalisation et de modernisation de la gestion opérationnelle des établissements hôteliers.

L'objectif principal consistait à concevoir et implémenter une plateforme complète couvrant l'ensemble du cycle hôtelier : la réservation en ligne avec visualisation 3D interactive des chambres, le paiement sécurisé via Stripe, la gestion des arrivées et départs (check-in/check-out), et l'assistance aux clients par un concierge intelligent basé sur l'IA. Le système implique trois acteurs distincts : le Client, le Réceptionniste et l'Administrateur.

La démarche adoptée s'appuie sur une analyse fonctionnelle approfondie suivie d'une conception UML rigoureuse, puis d'une implémentation utilisant Laravel 12 (PHP 8.2) pour le backend et Vue.js 3 pour le frontend. L'architecture MVC retenue garantit la séparation des responsabilités et facilite la maintenance évolutive.

Les résultats obtenus démontrent une solution robuste, sécurisée et innovante répondant efficacement aux besoins de digitalisation du secteur hôtelier.

Mots-clés : Gestion hôtelière, Laravel, Vue.js, Réservation en ligne, Visualisation 3D, Intelligence artificielle, Paiement en ligne.

Abstract

This report details the development of **Grand Palais**, an intelligent hotel reservation web application carried out as part of our final year project at the Mathematics and Computer Science department of the Higher School of Technology Fkih Ben Salah. This project addresses the digitalization and modernization of operational management in the hospitality industry.

The main objective was to design and implement a complete platform covering the full hotel lifecycle : online room reservation with interactive 3D visualization, secure payment via Stripe, check-in and check-out management, and AI-powered concierge assistance via WhatsApp. The system involves three distinct actors : the Client, the Receptionist, and the Administrator.

The approach is based on thorough functional analysis followed by rigorous UML design, then implementation using Laravel 12 (PHP 8.2) for the backend and Vue.js 3 with Vite for the frontend, connected to a MySQL database.

Keywords : Hotel management, Laravel, Vue.js, Online booking, 3D visualization, Artificial intelligence, Online payment.

Liste des Acronymes

TABLE 1 – Liste des acronymes utilisés dans le rapport

Acronyme	Signification
AI	Artificial Intelligence (Intelligence Artificielle)
API	Application Programming Interface
GLTF	GL Transmission Format (format de modèles 3D)
HTTP	HyperText Transfer Protocol
IA	Intelligence Artificielle
JWT	JSON Web Token
MVC	Modèle-Vue-Contrôleur
ORM	Object-Relational Mapping
PFE	Projet de Fin d'Études
PMS	Property Management System (Système de Gestion Hôtelière)
QR	Quick Response (code-barres 2D)
REST	Representational State Transfer
SPA	Single Page Application
SQL	Structured Query Language
UML	Unified Modeling Language
URL	Uniform Resource Locator
USMS	Université Sultan Moulay Slimane

Table des matières

Dédicace	i
Remerciements	ii
Résumé	iii
Abstract	iv
Liste des Acronymes	v
Table des figures	ix
Liste des tableaux	x
Introduction Générale	1
1 Étude Préalable	2
1.1 Introduction	2
1.2 Contexte Général du Projet	2
1.2.1 Présentation du contexte hôtelier	2
1.2.2 Présentation du projet	2
1.3 Problématique et Analyse de l'Existant	2
1.3.1 Problématique	2
1.3.2 Étude de l'existant	3
1.3.3 Critique de l'existant	3
1.4 Solution Proposée	3
1.4.1 Spécification des besoins	3
1.4.1.1 Besoins fonctionnels	3
1.4.1.2 Besoins non fonctionnels	4
1.4.2 Identification des acteurs	4
1.5 Conclusion	5
2 Conception	6
2.1 Introduction	6
2.2 Méthodologie de développement	6

2.2.1	Choix de la méthodologie itérative	6
2.2.2	Cycles de développement	6
2.3	Analyse fonctionnelle	7
2.3.1	Diagrammes de cas d'utilisation	7
2.3.1.1	Diagramme de cas d'utilisation — Client	7
2.3.1.2	Diagramme de cas d'utilisation — Réceptionniste	8
2.3.1.3	Diagramme de cas d'utilisation — Administrateur	9
2.3.2	Diagrammes de séquence	9
2.3.2.1	Inscription et connexion du client	10
2.3.2.2	Réservation 3D et paiement en ligne	11
2.3.2.3	Check-in et Check-out à la réception	12
2.3.3	Modélisation des données	12
2.3.3.1	Diagrammes de classe par section	12
2.3.3.2	Description des entités principales	14
2.4	Conclusion	15
3	Réalisation	16
3.1	Introduction	16
3.2	Architecture du système	16
3.2.1	Implémentation de l'architecture MVC	16
3.3	Langages et Outils de Développement	17
3.3.1	Langages de Programmation	17
3.3.1.1	PHP 8.2	17
3.3.1.2	JavaScript (ES2023) / TypeScript	17
3.3.2	Frameworks et Bibliothèques	18
3.3.2.1	Laravel 12 (Backend)	18
3.3.2.2	Vue.js 3 + Vite (Frontend)	18
3.3.3	Gestion des Données	18
3.3.3.1	MySQL	18
3.3.3.2	Eloquent ORM	19
3.3.4	Sécurité et Authentification	19
3.3.5	Outils de Développement	19
3.4	Interfaces	20
3.4.1	Interfaces Communes	20
3.4.1.1	Page d'accueil	20
3.4.1.2	Page de connexion et d'inscription	21
3.4.1.3	Page de profil client	22
3.4.2	Interface Client — Réservation 3D	23
3.4.2.1	Visualisation 3D et sélection de chambre	23

3.4.2.2	Paielement en ligne (Stripe)	24
3.4.3	Interface Réceptionniste	25
3.4.4	Interfaces Administrateur	26
3.4.4.1	Tableau de bord administratif	26
3.4.4.2	Gestion des chambres et des utilisateurs	27
3.5	Conclusion	27
Conclusion Générale		28
A Code Source		30
B Scripts SQL		31
B.1	Création de la base de données	31
B.2	Tables principales	31

Table des figures

2.1	Diagramme de cas d'utilisation — Client	7
2.2	Diagramme de cas d'utilisation — Réceptionniste	8
2.3	Diagramme de cas d'utilisation — Administrateur	9
2.4	Diagramme de séquence — Inscription, validation et connexion client .	10
2.5	Diagramme de séquence — Recherche 3D, réservation et paiement Stripe	11
2.6	Diagramme de séquence — Check-in et Check-out (réceptionniste) . . .	12
2.7	Diagramme de classe — Utilisateurs et Structure	13
2.8	Diagramme de classe global du système	14
3.1	Architecture MVC découplée du système Grand Palais	17
3.2	Page d'accueil de l'application Grand Palais	20
3.3	Interface de connexion et d'inscription	21
3.4	Interface profil utilisateur	22
3.5	Maquette 3D interactive — sélection de chambre	23
3.6	Interface de paiement sécurisé via Stripe	24
3.7	Tableau de bord du réceptionniste	25
3.8	Tableau de bord administratif	26
3.9	Interface de gestion des chambres	27

Liste des tableaux

1	Liste des acronymes utilisés dans le rapport	v
1.1	Acteurs du système Grand Palais et leurs responsabilités	4
2.1	Entités principales de la base de données Grand Palais	15

Introduction Générale

La transformation numérique du secteur de l'hôtellerie et du tourisme s'impose aujourd'hui comme une nécessité incontournable. Face à une clientèle de plus en plus connectée et exigeante, les hôtels doivent moderniser leurs outils de gestion pour rester compétitifs, améliorer l'expérience client et optimiser leur efficacité opérationnelle.

Les systèmes de gestion hôtelière traditionnels reposent souvent sur des processus manuels et des outils fragmentés (tableaux Excel, registres papier, logiciels isolés) qui génèrent des inefficacités, des erreurs et une mauvaise expérience client. La réservation en ligne, la gestion des disponibilités en temps réel et la communication personnalisée avec les clients sont devenus des critères essentiels de compétitivité.

C'est dans ce contexte que s'inscrit notre Projet de Fin d'Études : la conception et le développement de **Grand Palais**, une application web de gestion hôtelière moderne et intelligente. La problématique centrale est la suivante : *comment concevoir une plateforme numérique unifiée qui digitalise l'intégralité du parcours client — de la réservation en ligne jusqu'au départ de l'hôtel — tout en offrant des outils d'administration performants aux équipes internes ?*

Ce mémoire s'organise autour de trois chapitres structurés :

- Le **premier chapitre** présente l'étude préalable : contexte du projet, analyse de l'existant et spécification des besoins fonctionnels et non fonctionnels.
- Le **deuxième chapitre** développe la conception du système à travers une modélisation UML complète (cas d'utilisation, séquences, classes).
- Le **troisième chapitre** expose la réalisation technique, présentant les choix technologiques retenus et les principales interfaces de l'application.

Chapitre 1

Étude Préalable

1.1 Introduction

L'étude préalable constitue la première étape fondamentale du processus de développement logiciel. Elle permet de comprendre l'environnement dans lequel le système va s'intégrer, d'identifier les besoins réels des utilisateurs et de définir clairement les contours fonctionnels et techniques du projet.

1.2 Contexte Général du Projet

1.2.1 Présentation du contexte hôtelier

L'industrie hôtelière mondiale connaît une mutation profonde sous l'effet de la digitalisation. Les voyageurs réservent désormais leurs séjours principalement en ligne, comparent les offres et s'attendent à une expérience fluide et personnalisée. Les hôtels qui ne disposent pas d'un système de gestion numérique intégré perdent en compétitivité et en qualité de service. Notre projet **Grand Palais** vise à répondre à cette demande.

1.2.2 Présentation du projet

Grand Palais est une plateforme web complète de gestion hôtelière (PMS — Property Management System) développée dans le cadre du PFE. Elle centralise la gestion des réservations, des chambres, des paiements et de la communication client en un seul outil moderne, accessible depuis tout navigateur web.

1.3 Problématique et Analyse de l'Existant

1.3.1 Problématique

Les hôtels de taille moyenne gèrent souvent leurs réservations de manière fragmentée : réservations par téléphone reportées manuellement, fichiers Excel pour les disponibilités,

factures manuscrites et aucune visibilité en temps réel sur le taux d'occupation. Cette approche génère des erreurs (doubles réservations, pertes de données), une mauvaise expérience client et une charge administrative élevée pour le personnel.

1.3.2 Étude de l'existant

Les solutions existantes se répartissent en deux catégories :

Systèmes traditionnels Gestion par registres papier ou tableurs Excel. Aucune visibilité temps réel, risque de doubles réservations, pas de traçabilité.

Logiciels propriétaires Des solutions comme Opera PMS ou Protel existent, mais elles sont coûteuses, complexes à déployer et peu adaptées aux petits et moyens établissements.

Plateformes OTA Des sites comme Booking.com ou Airbnb permettent la réservation en ligne, mais prélèvent des commissions élevées (15–25%) et ne fournissent pas d'outils de gestion interne.

1.3.3 Critique de l'existant

L'analyse des solutions existantes révèle plusieurs lacunes majeures :

- Absence de visualisation interactive des chambres disponibles ;
- Aucune intégration d'intelligence artificielle pour l'assistance client ;
- Coût prohibitif des solutions propriétaires pour les établissements indépendants ;
- Pas de gestion intégrée du parcours client de bout en bout ;
- Manque de tableau de bord analytique en temps réel pour les gestionnaires.

1.4 Solution Proposée

1.4.1 Spécification des besoins

1.4.1.1 Besoins fonctionnels

La plateforme **Grand Palais** doit couvrir les fonctionnalités suivantes :

- Permettre aux clients de s'inscrire, se connecter et gérer leur profil ;
- Offrir une visualisation 3D interactive des chambres avec disponibilité en temps réel (chambres libres en vert, occupées en rouge) ;
- Gérer le cycle complet de réservation : sélection, options de services additionnels, application de codes promotionnels et paiement en ligne ;
- Envoyer un e-mail de confirmation avec QR Code après chaque réservation ;

- Fournir un assistant concierge IA via WhatsApp pour répondre aux questions des clients sur leur séjour ;
- Permettre au réceptionniste de gérer les arrivées (check-in via QR Code) et les départs (check-out avec génération de facture) ;
- Offrir à l'administrateur un tableau de bord complet : gestion des chambres, des utilisateurs, des statistiques et envoi de campagnes e-mail.

1.4.1.2 Besoins non fonctionnels

Sécurité Authentification via Laravel Sanctum (tokens API), contrôle d'accès par rôles (Spatie Permission), chiffrement des mots de passe (bcrypt).

Performance Architecture SPA avec Vue.js 3 pour des transitions fluides, API REST optimisée avec pagination et indexation de la base de données.

Ergonomie Interface premium responsive, accessible sur tous les appareils (ordinateur, tablette, smartphone).

Fiabilité Paiements sécurisés via l'API Stripe ; gestion des erreurs et des transactions échouées.

Extensibilité Architecture modulaire facilitant l'ajout de nouvelles fonctionnalités (application mobile, multi-hôtels, etc.).

1.4.2 Identification des acteurs

Le système Grand Palais implique trois acteurs principaux, présentés dans le tableau 1.1.

TABLE 1.1 – Acteurs du système Grand Palais et leurs responsabilités

Acteur	Rôle et responsabilités
Client	Visiteur authentifié qui recherche des chambres, effectue des réservations en ligne, paie via Stripe, et interagit avec le concierge IA sur WhatsApp.
Réceptionniste	Agent de l'hôtel qui gère les arrivées (check-in par scan QR), les départs (check-out avec facturation), et le suivi des réservations en cours.
Administrateur	Gestionnaire qui administre le catalogue des chambres, les comptes utilisateurs, les codes promotionnels, les statistiques et les campagnes de communication par e-mail.

1.5 Conclusion

Cette étude préalable nous a permis de cerner précisément le contexte hôtelier, d'analyser les limites des solutions existantes et de définir un ensemble cohérent de besoins fonctionnels et non fonctionnels. Le système Grand Palais se positionne comme une solution innovante, complète et accessible, répondant aux attentes des trois acteurs identifiés. Ces fondations solides guident la conception UML détaillée que nous présentons dans le chapitre suivant.

Chapitre 2

Conception

2.1 Introduction

Ce chapitre présente la démarche de conception du système Grand Palais. Nous y exposons la méthodologie choisie, l'analyse fonctionnelle traduite en diagrammes UML (cas d'utilisation, séquences), ainsi que la modélisation des données qui constituent la base de notre architecture logicielle.

2.2 Méthodologie de développement

2.2.1 Choix de la méthodologie itérative

Face à la richesse fonctionnelle du projet et à la nécessité d'intégrer des technologies variées (3D, IA, paiement en ligne), nous avons adopté une approche de développement itérative et incrémentale pour les raisons suivantes :

- Validation progressive de chaque module avec des tests fonctionnels ;
- Flexibilité pour ajuster les fonctionnalités en cours de développement ;
- Livraison rapide de prototypes permettant de détecter les problèmes tôt ;
- Meilleure gestion des risques liés aux intégrations tierces (Stripe, Twilio).

2.2.2 Cycles de développement

Le développement a été organisé en quatre itérations principales :

1. **Itération 1** : Authentification, gestion des rôles et des profils ;
2. **Itération 2** : Catalogue des chambres, visualisation 3D et réservation ;
3. **Itération 3** : Paiement Stripe, QR Code, concierge IA (WhatsApp) ;
4. **Itération 4** : Interfaces réceptionniste, tableau de bord admin et campagnes.

2.3 Analyse fonctionnelle

2.3.1 Diagrammes de cas d'utilisation

Pour modéliser les interactions entre les acteurs et le système, nous avons élaboré des diagrammes de cas d'utilisation UML couvrant chaque profil utilisateur.

2.3.1.1 Diagramme de cas d'utilisation — Client

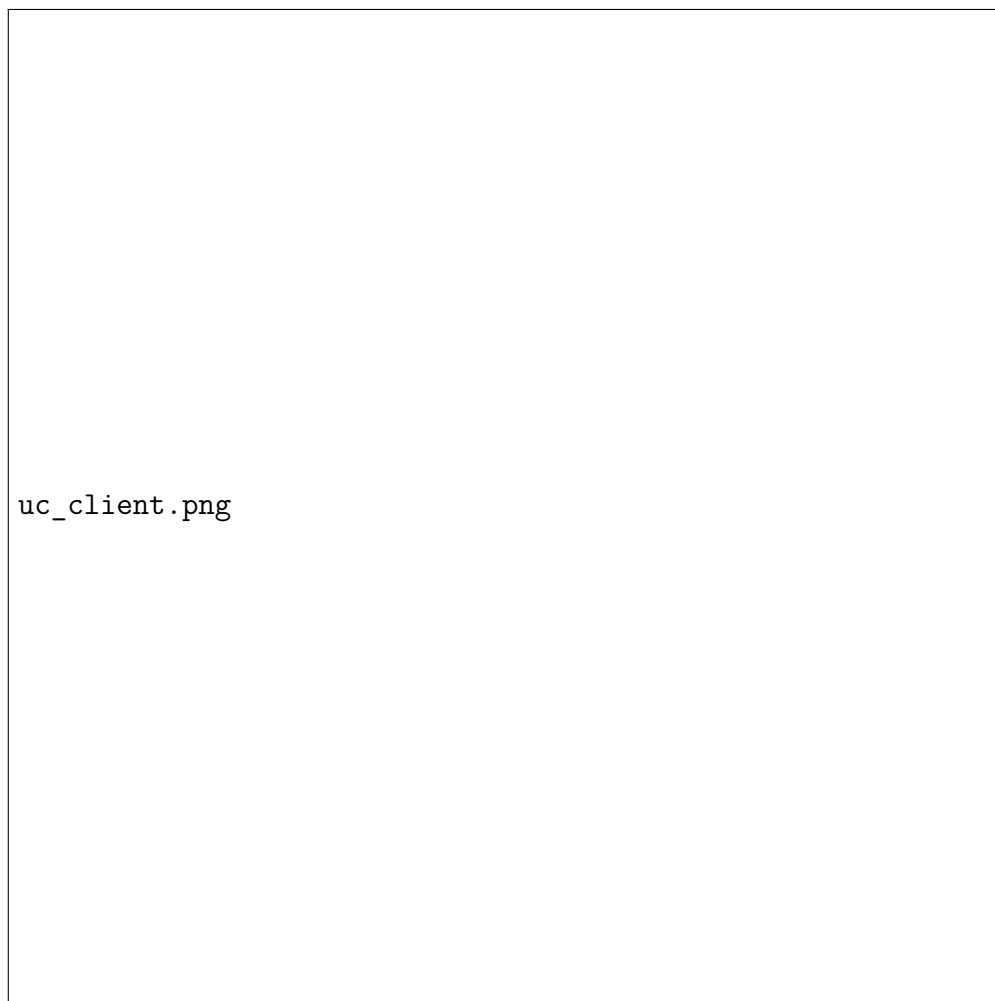


FIGURE 2.1 – Diagramme de cas d'utilisation — Client

Le client peut s'inscrire, se connecter, consulter les chambres disponibles via la maquette 3D, effectuer une réservation, payer en ligne, gérer ses réservations et interagir avec le concierge IA sur WhatsApp.

2.3.1.2 Diagramme de cas d'utilisation — Réceptionniste



FIGURE 2.2 – Diagramme de cas d'utilisation — Réceptionniste

Le réceptionniste peut rechercher une réservation par nom ou QR Code, valider le check-in, gérer le check-out et générer la facture finale du client.

2.3.1.3 Diagramme de cas d'utilisation — Administrateur

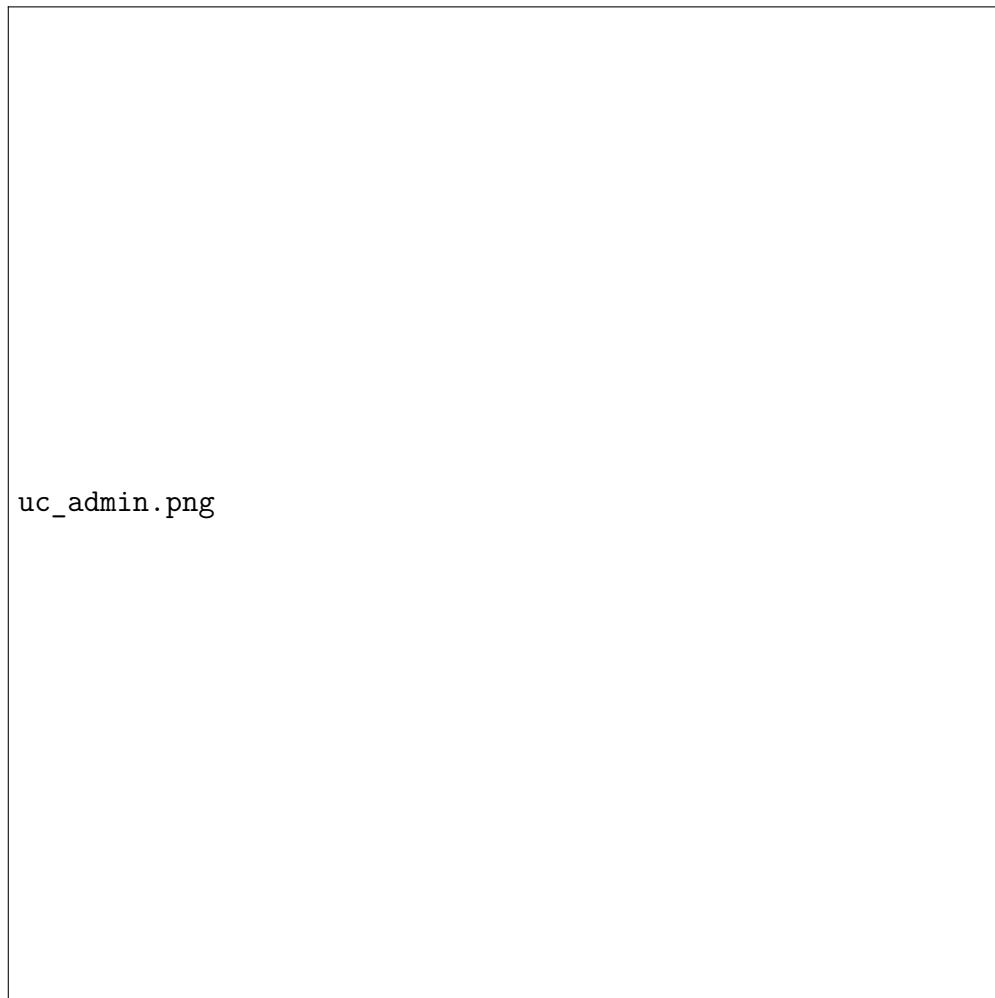


FIGURE 2.3 – Diagramme de cas d'utilisation — Administrateur

L'administrateur gère l'ensemble du back-office : création et modification des chambres, gestion des utilisateurs et de leurs rôles, création de codes promotionnels, consultation des statistiques et envoi de campagnes e-mail à la base clients.

2.3.2 Diagrammes de séquence

Les diagrammes de séquence illustrent les flux d'interaction entre les acteurs et le système pour chaque scénario métier principal.

2.3.2.1 Inscription et connexion du client

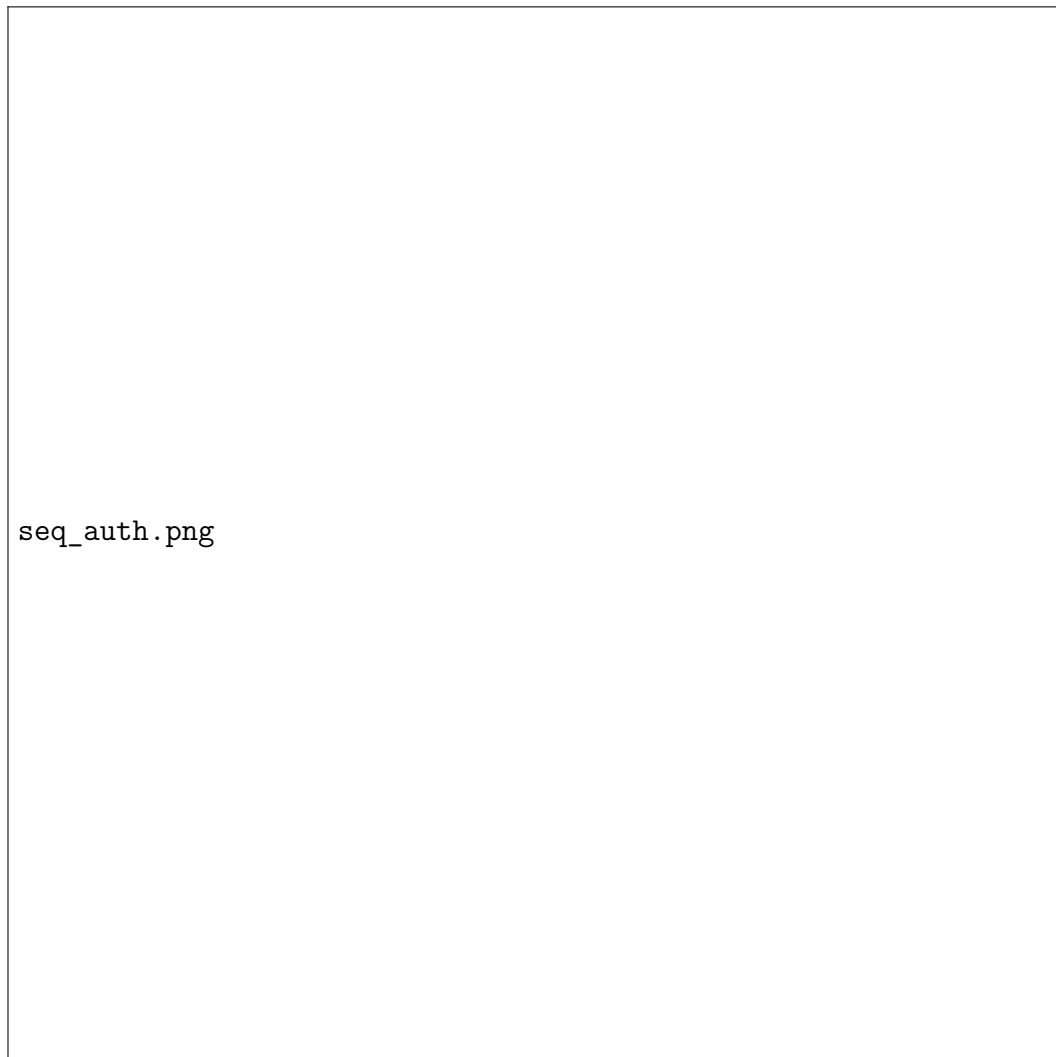


FIGURE 2.4 – Diagramme de séquence — Inscription, validation et connexion client

Le client remplit le formulaire d'inscription ; le système vérifie les données et envoie un e-mail de validation. Après confirmation, le client peut se connecter et accéder à son espace personnel.

2.3.2.2 Réservation 3D et paiement en ligne



FIGURE 2.5 – Diagramme de séquence — Recherche 3D, réservation et paiement Stripe

Le client sélectionne ses dates, visualise les chambres disponibles sur la maquette 3D, ajoute des services, applique un code promotionnel, puis paie via Stripe. Un e-mail de confirmation contenant un QR Code est envoyé automatiquement.

2.3.2.3 Check-in et Check-out à la réception

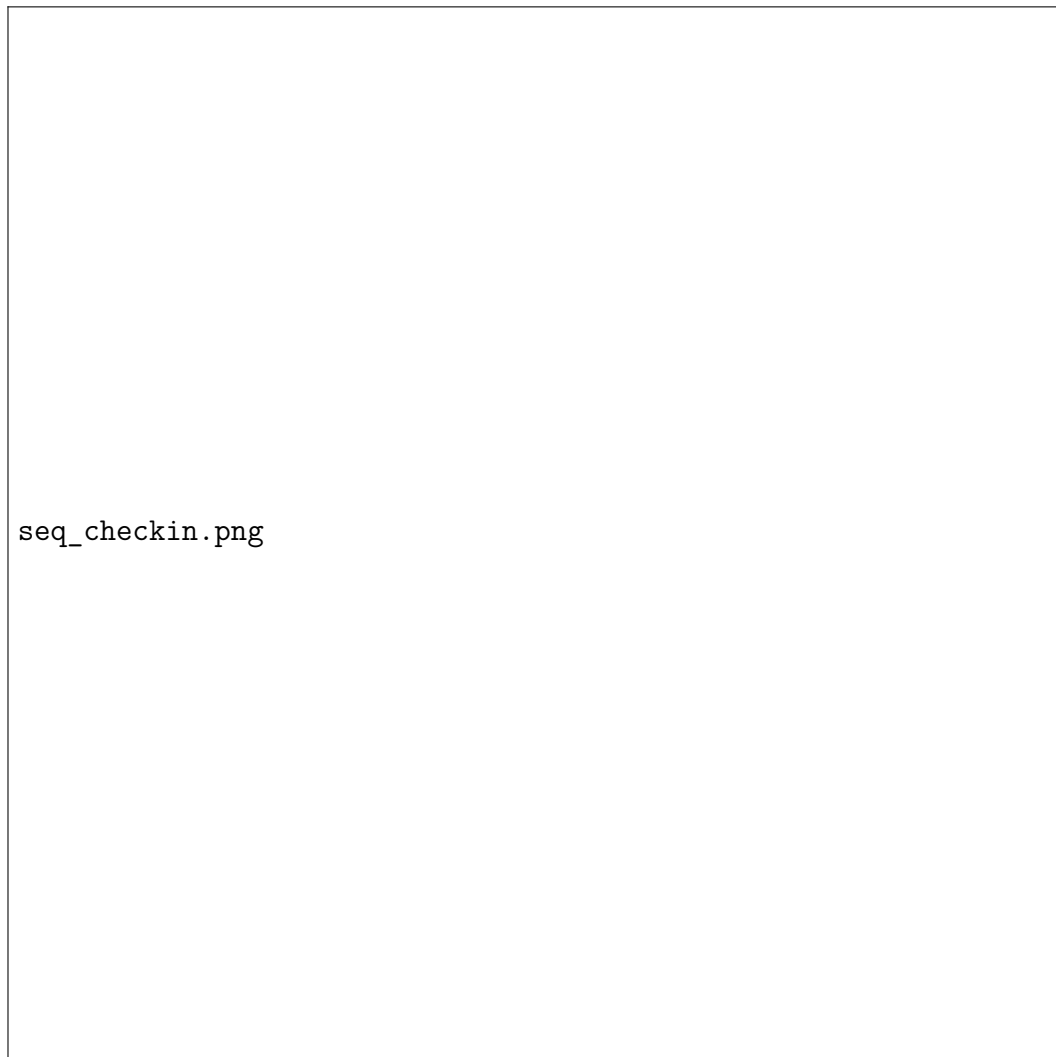


FIGURE 2.6 – Diagramme de séquence — Check-in et Check-out (réceptionniste)

Le réceptionniste scanne le QR Code du client pour retrouver sa réservation et valider son arrivée. Au départ, il génère la facture finale, enregistre le check-out et libère la chambre pour le service de nettoyage.

2.3.3 Modélisation des données

2.3.3.1 Diagrammes de classe par section

Pour construire la base de données, nous avons conçu des diagrammes de classe qui représentent les entités métier et leurs relations.



FIGURE 2.7 – Diagramme de classe — Utilisateurs et Structure

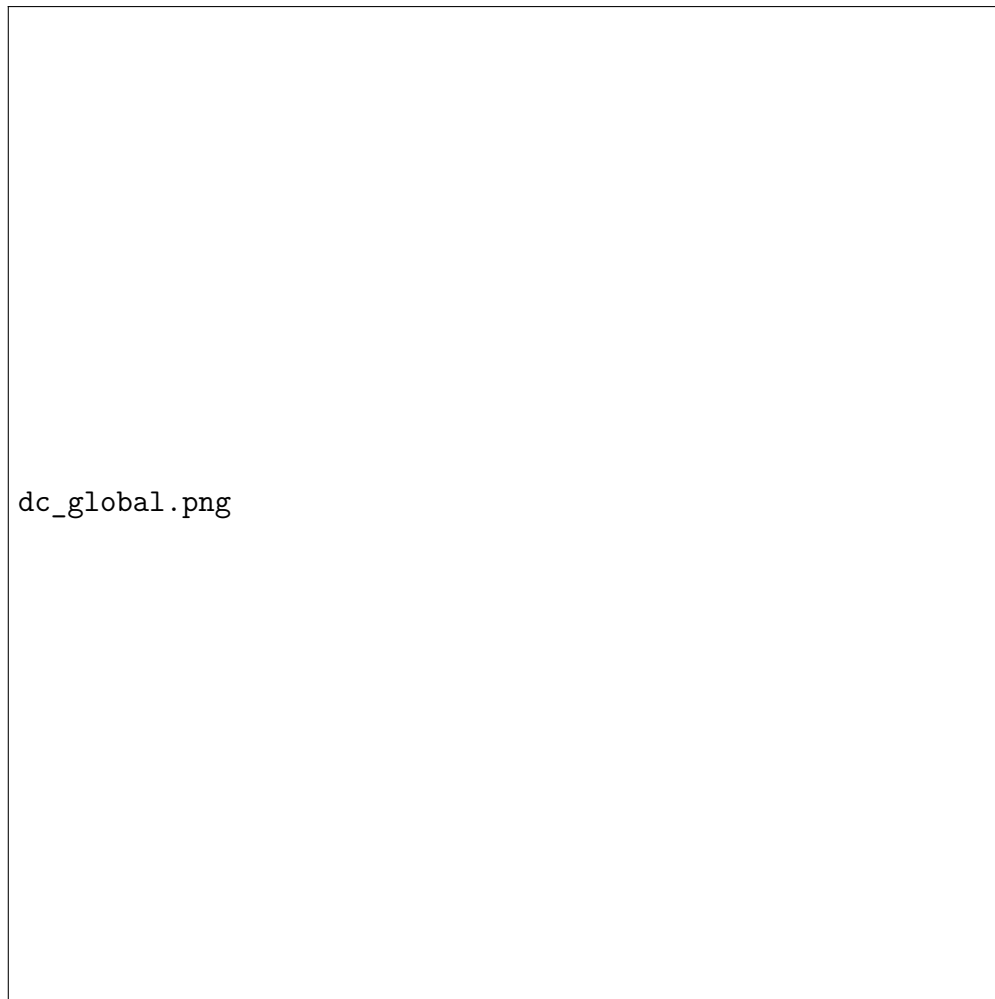


FIGURE 2.8 – Diagramme de classe global du système

2.3.3.2 Description des entités principales

Le tableau 2.1 décrit les entités centrales de la base de données du système Grand Palais.

TABLE 2.1 – Entités principales de la base de données Grand Palais

Entité (Table)	Description
<code>users</code>	Comptes de tous les utilisateurs (clients, réceptionnistes, administrateurs) avec authentification, rôles, photo de profil et informations personnelles (téléphone, adresse, pays).
<code>rooms</code>	Catalogue des chambres avec numéro, étage, surface, vue, statut (disponible, occupée, maintenance) et prix.
<code>room_types</code>	Types de chambres (Standard, Deluxe, Suite) avec description, capacité et tarif de base.
<code>reservations</code>	Réservations avec dates de séjour, statut (pending, confirmed, checked_in, checked_out, cancelled), prix total, remise appliquée et QR Code associé.
<code>payments</code>	Transactions de paiement liées à une réservation avec montant, devise, méthode (carte Stripe, sur place) et statut.
<code>promotions</code>	Codes promotionnels avec pourcentage ou montant de réduction, dates de validité et nombre d'utilisations.
<code>services</code>	Services additionnels proposés à la réservation (petit-déjeuner, spa, transfert aéroport, etc.) avec tarification.

2.4 Conclusion

Dans ce chapitre, nous avons présenté la conception complète du système à travers une analyse fonctionnelle rigoureuse et une modélisation UML structurée. Cette conception détaillée prépare efficacement la phase de réalisation technique.

Chapitre 3

Réalisation

3.1 Introduction

Après avoir finalisé l'analyse fonctionnelle ainsi que la conception de cette application, nous présentons ici les choix techniques retenus pour son développement. Ce chapitre explique l'environnement de travail utilisé, les technologies sélectionnées et les principales fonctionnalités à travers des captures d'écran.

3.2 Architecture du système

3.2.1 Implémentation de l'architecture MVC

Le système Grand Palais repose sur une architecture **MVC découplée** : le backend Laravel expose une API REST et le frontend Vue.js consomme cette API de façon indépendante.

Modèle Couche de données gérée par l'ORM Eloquent de Laravel avec MySQL comme SGBD. Chaque table de la base de données correspond à un modèle Eloquent avec ses relations (`hasMany`, `belongsTo`).

Vue SPA (Single Page Application) développée avec Vue.js 3. Les composants Vue communiquent avec le backend via des requêtes HTTP (Axios) vers les endpoints de l'API REST.

Contrôleur Contrôleurs Laravel qui reçoivent les requêtes HTTP, appliquent la logique métier et retournent des réponses JSON consommées par le frontend.



FIGURE 3.1 – Architecture MVC découplée du système Grand Palais

3.3 Langages et Outils de Développement

Le développement a impliqué plusieurs langages, frameworks et outils, sélectionnés selon leur fiabilité, leurs performances et leur adéquation au projet.

3.3.1 Langages de Programmation

3.3.1.1 PHP 8.2

PHP est le langage côté serveur utilisé pour le backend. Sa version 8.2 apporte des fonctionnalités modernes (types enum, fibers, attributs) qui améliorent la lisibilité et la robustesse du code. Il est exécuté via le serveur Apache de XAMPP.

3.3.1.2 JavaScript (ES2023) / TypeScript

JavaScript est le langage du frontend. Utilisé avec Vue.js 3 et le bundler Vite, il permet de créer une interface réactive et fluide sous forme de SPA. Les configurations

Vite utilisent des modules ES natifs pour des builds ultra-rapides.

3.3.2 Frameworks et Bibliothèques

3.3.2.1 Laravel 12 (Backend)

Laravel est le framework PHP choisi pour le backend. Il fournit un routage puissant, un ORM complet (Eloquent), une gestion des files d'attente (Queues), un système de mails (Mailable) et une intégration native avec Stripe et Twilio. Les packages complémentaires utilisés sont :

- **Spatie Permission** : gestion des rôles et permissions (admin, receptionist, client) ;
- **Laravel Sanctum** : authentification par tokens API pour la SPA ;
- **Laravel Socialite** : connexion via comptes sociaux ;
- **Simple QRCode** : génération des QR Codes de confirmation ;
- **DomPDF** : génération de factures PDF ;
- **OpenAI PHP Client** : intégration du concierge IA.

3.3.2.2 Vue.js 3 + Vite (Frontend)

Vue.js 3 est le framework JavaScript progressif utilisé pour le frontend. L'architecture en composants `.vue` (SFC — Single File Components) assure la séparation claire entre template, logique et style. Les bibliothèques complémentaires intégrées sont :

- **Vue Router 4** : navigation entre les vues de la SPA ;
- **Pinia** : gestion d'état globale de l'application ;
- **Axios** : communication avec l'API REST Laravel ;
- **Three.js** : rendu 3D interactif de la maquette de l'hôtel (GLTF) ;
- **Stripe.js** : intégration du formulaire de paiement sécurisé ;
- **Chart.js / vue-chartjs** : graphiques statistiques du tableau de bord ;
- **SweetAlert2 / vue3-toastify** : notifications et confirmations.

3.3.3 Gestion des Données

3.3.3.1 MySQL

MySQL est le SGBD relationnel utilisé pour persister l'ensemble des données du système. Il est hébergé localement via XAMPP. La base de données contient neuf tables principales liées par des clés étrangères avec intégrité référentielle.

3.3.3.2 Eloquent ORM

Eloquent est l'ORM natif de Laravel. Il traduit les opérations SQL en méthodes PHP orientées objet, simplifiant les requêtes complexes grâce aux relations (**hasMany**, **belongsToMany**) et aux scopes.

3.3.4 Sécurité et Authentification

L'authentification est gérée par **Laravel Sanctum** qui émet des tokens d'API sécurisés stockés côté client. Le contrôle d'accès par rôles est assuré par **Spatie Permission** : chaque route API est protégée par un middleware vérifiant le rôle de l'utilisateur connecté (**admin**, **receptionist**, **client**). Les mots de passe sont chiffrés avec **bcrypt**.

3.3.5 Outils de Développement

IDE Visual Studio Code — environnement de développement principal avec extensions PHP Intelephense et Volar (Vue.js).

Modélisation UML Enterprise Architect — pour la conception des diagrammes de cas d'utilisation, de séquence et de classes.

Tests API Postman — pour tester et valider tous les endpoints REST.

Versioning Git / GitHub — gestion du code source et collaboration.

Serveur local XAMPP (Apache + MySQL) avec **php artisan serve**.

3.4 Interfaces

3.4.1 Interfaces Communes

3.4.1.1 Page d'accueil

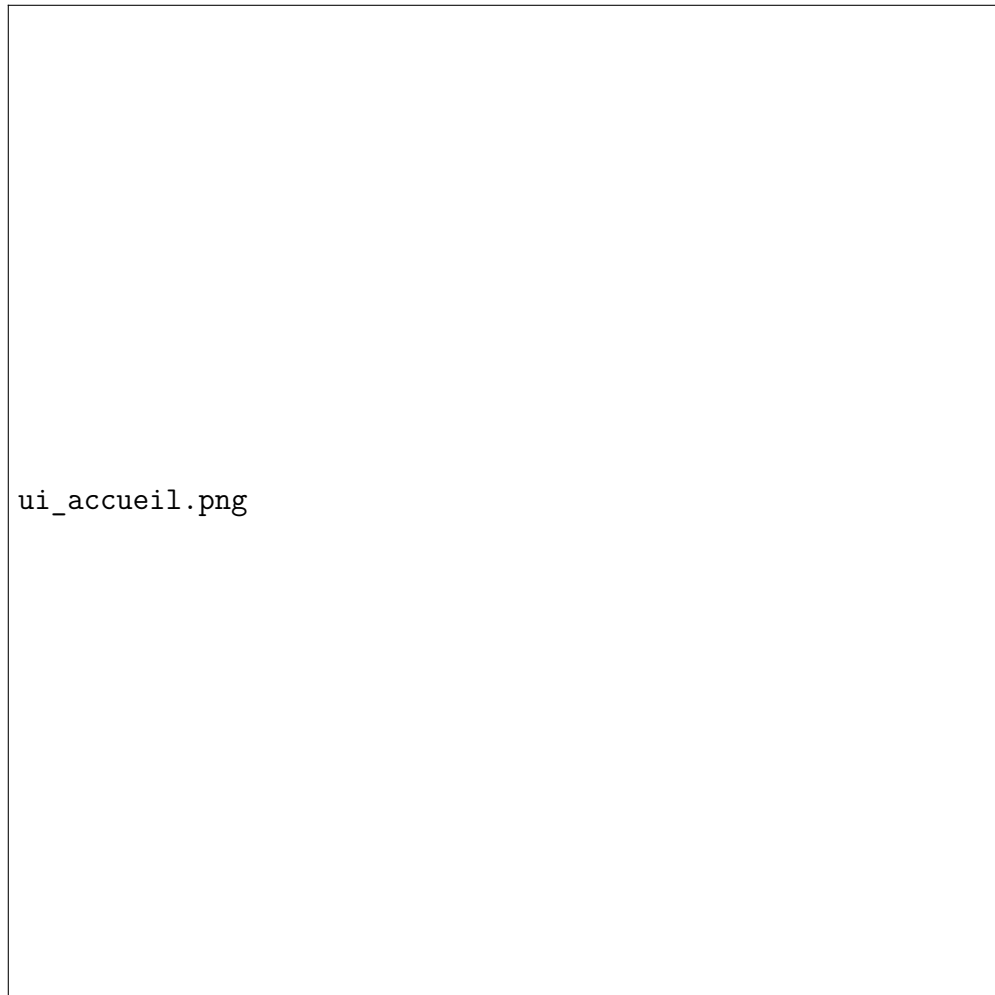


FIGURE 3.2 – Page d'accueil de l'application Grand Palais

La page d'accueil présente une interface premium avec une héro section immersive, une présentation des types de chambres disponibles, des services de l'hôtel et un appel à l'action vers la réservation en ligne.

3.4.1.2 Page de connexion et d'inscription

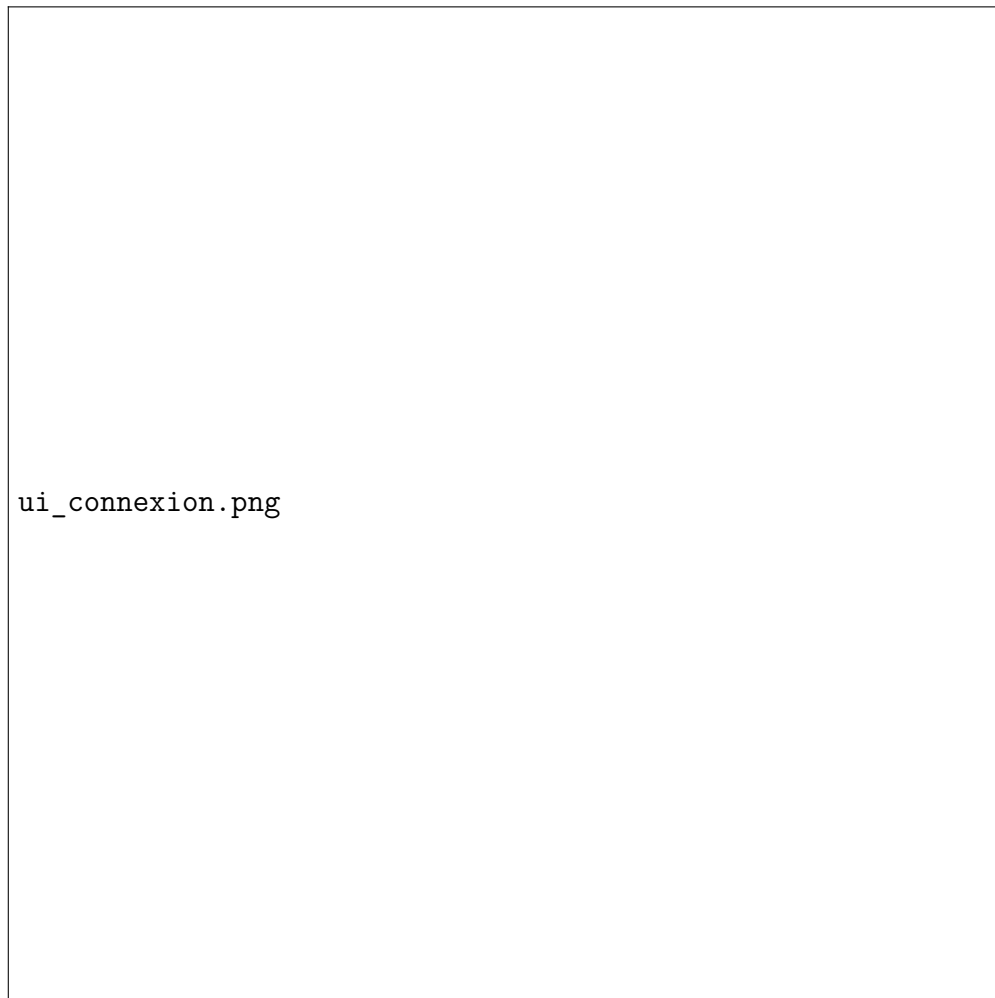
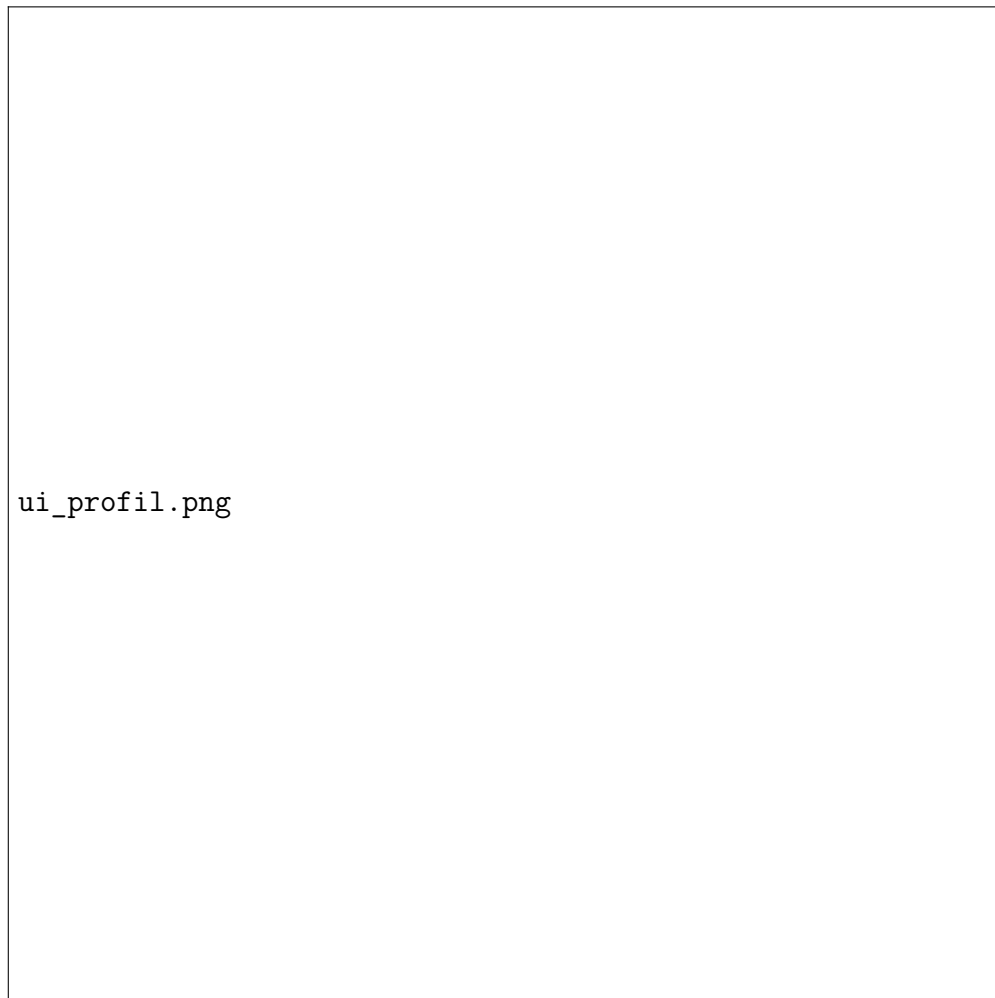


FIGURE 3.3 – Interface de connexion et d'inscription

Le client saisit ses identifiants pour se connecter. Lors de l'inscription, un e-mail de vérification est envoyé automatiquement. La connexion via réseaux sociaux est également disponible (Laravel Socialite).

3.4.1.3 Page de profil client



ui_profil.png

FIGURE 3.4 – Interface profil utilisateur

Le client peut modifier ses informations personnelles, changer sa photo de profil et consulter l'historique complet de ses réservations passées.

3.4.2 Interface Client — Réservation 3D

3.4.2.1 Visualisation 3D et sélection de chambre

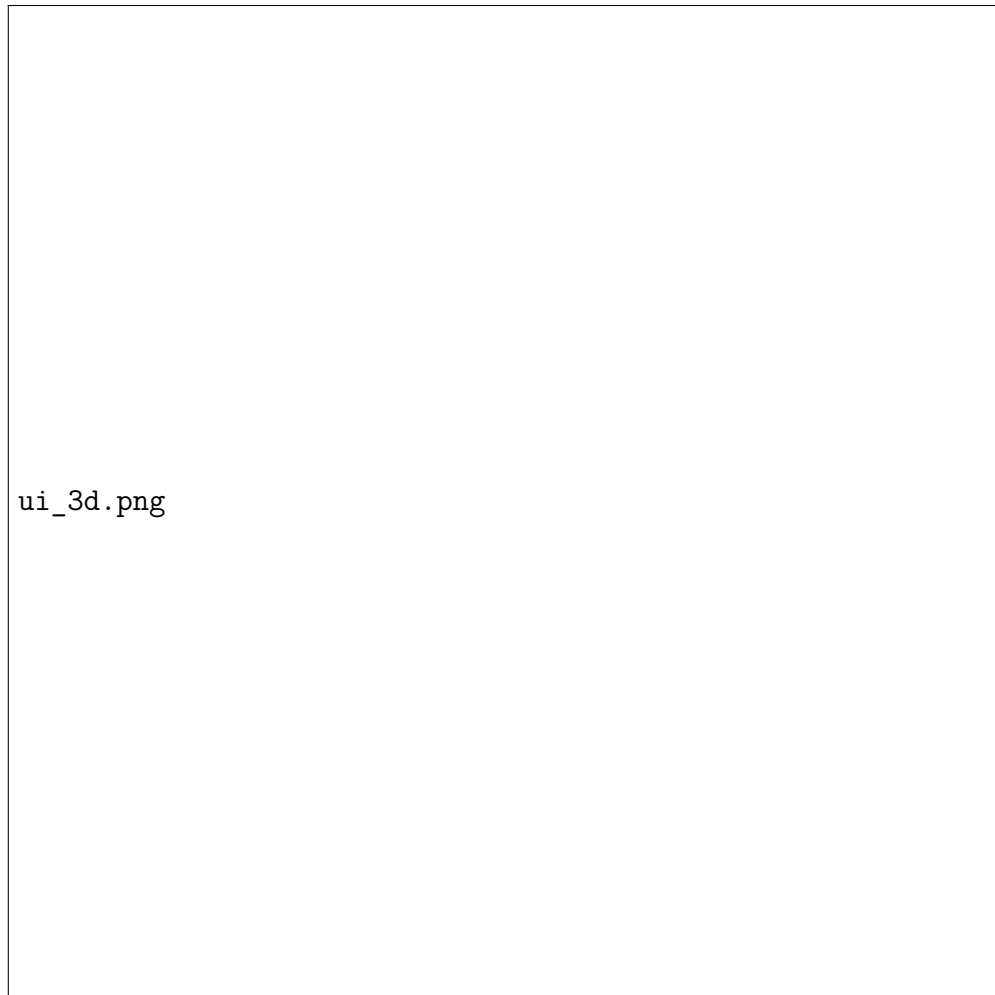


FIGURE 3.5 – Maquette 3D interactive — sélection de chambre

Après avoir saisi ses dates de séjour, le client visualise la maquette 3D rendue avec Three.js. Les chambres libres s’affichent en vert et les chambres occupées en rouge. Un clic sur une chambre affiche ses détails et permet de lancer la réservation.

3.4.2.2 Paiement en ligne (Stripe)



FIGURE 3.6 – Interface de paiement sécurisé via Stripe

Le client valide sa réservation et paie via le formulaire Stripe intégré. Un e-mail de confirmation avec QR Code est ensuite envoyé automatiquement.

3.4.3 Interface Réceptionniste



FIGURE 3.7 – Tableau de bord du réceptionniste

Le réceptionniste accède à la liste des réservations du jour, peut rechercher un client par nom ou scanner son QR Code pour le check-in, et génère la facture PDF lors du check-out.

3.4.4 Interfaces Administrateur

3.4.4.1 Tableau de bord administratif

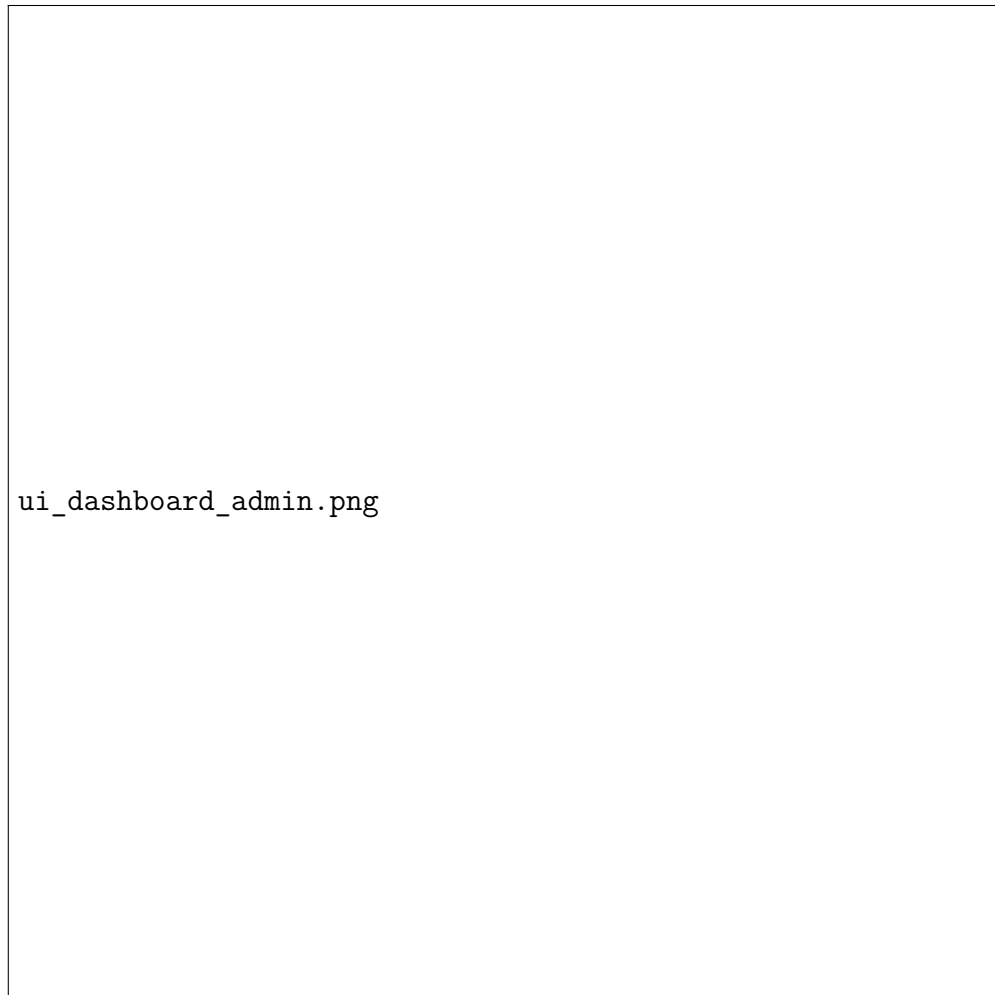


FIGURE 3.8 – Tableau de bord administratif

L'administrateur consulte les statistiques en temps réel (chiffre d'affaires, taux d'occupation) via des graphiques Chart.js.

3.4.4.2 Gestion des chambres et des utilisateurs



FIGURE 3.9 – Interface de gestion des chambres

L’administrateur crée et modifie les chambres du catalogue (type, étage, statut) et gère les comptes utilisateurs avec attribution de rôles.

3.5 Conclusion

La réalisation de Grand Palais a nécessité la maîtrise de technologies modernes et variées. Le choix de Laravel 12 et Vue.js 3 a permis de construire une architecture robuste, sécurisée et évolutive. Les intégrations uniques (Three.js, Stripe, Twilio, OpenAI) positionnent ce projet comme une solution innovante dans la gestion hôtelière.

Conclusion Générale

Synthèse du projet

Ce projet de fin d'études nous a permis de concevoir et de développer **Grand Palais**, une application web complète de gestion hôtelière moderne. La plateforme couvre l'intégralité du parcours client — de la réservation en ligne avec visualisation 3D jusqu'au départ — tout en offrant des outils de gestion performants au réceptionniste et à l'administrateur.

Apports du système développé

Valeur ajoutée pour l'hôtel

Le système Grand Palais apporte plusieurs avantages concrets :

- Disponibilités des chambres en temps réel via la maquette 3D interactive ;
- Automatisation du cycle de réservation avec paiement en ligne sécurisé ;
- Réduction de la charge administrative grâce au check-in/check-out numérique ;
- Communication client avancée via le concierge IA sur WhatsApp ;
- Tableaux de bord analytiques pour piloter l'activité en temps réel.

Fonctionnalités clés implémentées

Les fonctionnalités développées couvrent l'ensemble du cycle hôtelier : réservation 3D, paiement Stripe, confirmation QR Code, concierge IA WhatsApp, check-in/check-out réceptionniste, facturation PDF et campagnes e-mail clients.

Défis rencontrés et solutions apportées

Le développement a présenté plusieurs défis techniques majeurs :

Visualisation 3D Intégration de Three.js avec des modèles GLTF et synchronisation des couleurs des chambres avec les disponibilités en temps réel.

Paiement Stripe Configuration des webhooks et sécurisation du flux de confirmation de réservation après paiement.

Concierge IA Intégration de Twilio (WhatsApp) et de l'API OpenAI pour générer des réponses personnalisées basées sur le séjour du client.

Architecture SPA Gestion de l'authentification Sanctum entre le backend Laravel et le frontend Vue.js découplés sur des ports différents.

Compétences acquises

Ce projet nous a permis de développer et renforcer des compétences variées :

- **Compétences techniques** : Laravel, Vue.js, Three.js, Stripe, Twilio, OpenAI ;
- **Conception logicielle** : modélisation UML, architecture REST, gestion BDD ;
- **Gestion de projet** : planification itérative, versioning Git ;
- **Travail en équipe** : collaboration sur un dépôt GitHub commun.

Perspectives d'évolution

Plusieurs pistes d'amélioration peuvent être envisagées :

- Développement d'une application mobile (iOS/Android) pour les clients ;
- Support multi-hôtels pour gérer plusieurs établissements ;
- Module de tarification dynamique selon la saison et l'occupation ;
- Système de fidélité avancé avec points et récompenses ;
- Extension du concierge IA pour traiter les commandes de services en chambre.

Annexe A

Code Source

Le code source complet du projet est disponible sur GitHub à l'adresse suivante :

<https://github.com/mohsinedahnaoui/new-pfe>

Le repository contient la structure complète du projet avec :

- **Backend Laravel** (racine du dépôt) :
 - `app/Http/Controllers/` — Contrôleurs REST API (chambres, réservations, paiements, utilisateurs) ;
 - `app/Models/` — Modèles Eloquent (User, Room, Reservation, Payment, Promotion...);
 - `database/migrations/` — Scripts de création des tables MySQL ;
 - `routes/api.php` — Définition de toutes les routes REST protégées par Sanctum.
- **Frontend Vue.js** (/frontend/) :
 - `src/views/` — Pages principales (accueil, réservation 3D, dashboard admin, réception) ;
 - `src/components/` — Composants réutilisables (maquette 3D, formulaires, tableaux) ;
 - `src/stores/` — Stores Pinia (authentification, réservations, chambres) ;
 - `src/router/` — Configuration du routage Vue Router avec guards d'authentification.

Annexe B

Scripts SQL

B.1 Création de la base de données

```
1 CREATE DATABASE IF NOT EXISTS grand_palais
2   CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
3
4 USE grand_palais;
```

Listing B.1 – Création de la base de données Grand Palais

B.2 Tables principales

```
1 CREATE TABLE users (
2   id                BIGINT AUTO_INCREMENT PRIMARY KEY,
3   name              VARCHAR(255) NOT NULL,
4   email             VARCHAR(255) NOT NULL UNIQUE,
5   email_verified_at TIMESTAMP NULL,
6   password          VARCHAR(255) NULL,
7   phone             VARCHAR(255) NULL,
8   avatar            VARCHAR(255) NULL,
9   address            VARCHAR(255) NULL,
10  city               VARCHAR(255) NULL,
11  country            VARCHAR(255) NULL,
12  is_active          BIT(1) DEFAULT 1,
13  last_login_at      TIMESTAMP NULL,
14  created_at         TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
15  updated_at         TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
16    CURRENT_TIMESTAMP,
16  deleted_at         TIMESTAMP NULL
17 );
```

Listing B.2 – Table users (tous les utilisateurs)

```
1 CREATE TABLE rooms (
2   id                BIGINT AUTO_INCREMENT PRIMARY KEY,
```

```

3   room_type_id    BIGINT NOT NULL,
4   number          VARCHAR(255) NOT NULL UNIQUE,
5   floor           INT NOT NULL,
6   surface         DECIMAL(8,2),
7   view            VARCHAR(255) NULL,
8   status           ENUM('available','occupied','maintenance','blocked')
9                   DEFAULT 'available',
10  price_override  DECIMAL(10,2) NULL,
11  created_at       TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
12  FOREIGN KEY (room_type_id) REFERENCES room_types(id)
13 );
14
15 CREATE TABLE reservations (
16   id              BIGINT AUTO_INCREMENT PRIMARY KEY,
17   reference       VARCHAR(255) NOT NULL UNIQUE,
18   user_id         BIGINT NOT NULL,
19   room_id         BIGINT NOT NULL,
20   promotion_id    BIGINT NULL,
21   check_in        DATE NOT NULL,
22   check_out       DATE NOT NULL,
23   adults          INT DEFAULT 1,
24   children        INT DEFAULT 0,
25   status          ENUM('pending','confirmed','checked_in',
26                       'checked_out','cancelled','no_show')
27                   DEFAULT 'pending',
28   room_price      DECIMAL(10,2) NOT NULL,
29   services_price  DECIMAL(10,2) DEFAULT 0,
30   discount_amount DECIMAL(10,2) DEFAULT 0,
31   total_price     DECIMAL(10,2) NOT NULL,
32   qr_code        VARCHAR(255) NULL,
33   confirmed_at    TIMESTAMP NULL,
34   checked_in_at   TIMESTAMP NULL,
35   checked_out_at  TIMESTAMP NULL,
36   created_at      TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
37   FOREIGN KEY (user_id) REFERENCES users(id),
38   FOREIGN KEY (room_id) REFERENCES rooms(id)
39 );
40
41 CREATE TABLE payments (
42   id              BIGINT AUTO_INCREMENT PRIMARY KEY,
43   reservation_id  BIGINT NOT NULL,
44   user_id         BIGINT NOT NULL,
45   transaction_id  VARCHAR(255) UNIQUE NULL,
46   amount          DECIMAL(10,2) NOT NULL,
47   currency        VARCHAR(10) DEFAULT 'EUR',
48   method          ENUM('card','paypal','transfer','on_arrival','wallet'),

```

```
49  status          ENUM('pending','completed','failed','refunded')
50                      DEFAULT 'pending',
51  paid_at          TIMESTAMP NULL,
52  created_at       TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
53  FOREIGN KEY (reservation_id) REFERENCES reservations(id),
54  FOREIGN KEY (user_id) REFERENCES users(id)
55 );
```

Listing B.3 – Tables rooms et reservations